Project title:Applying Mixture of Recursions (MoR) to Vision Transformers (ViT)

Introduction:

- Vision Transformers (ViTs) achieve state-of-the-art results in computer vision but are computationally expensive due to their massive parameter counts.
- A fundamental inefficiency in ViTs is the **Uniform Depth Strategy**: simple background patches (like sky) are processed with the same deep computation as complex objects.
- I propose **MoR-ViT**, a recursive architecture that dynamically allocates compute. It processes important tokens deeply while allowing trivial ones to exit early, reducing both model size and FLOPs.

Motivation:

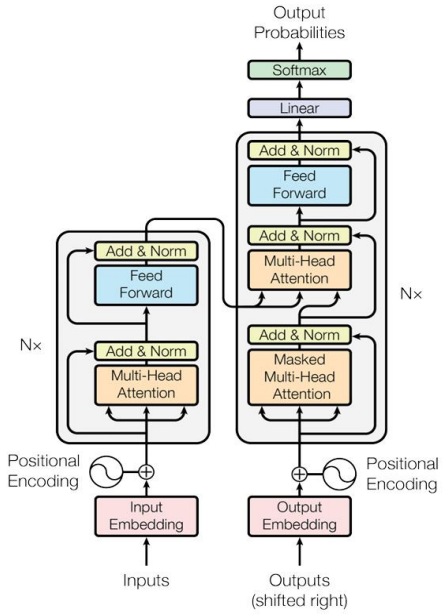
- In real-world images, information is not evenly distributed. Processing every pixel with the same cost is inefficient. We need "Adaptive Computation."
- Normal Vision Transformers are very large (more than 25 million parameters), so they run too slow on phones or small devices. We need a method where the model can go deeper without becoming too big
- Recent works like *Mixture-of-Recursions (MoR)* proved that recursive (shared weight) models work well on Transformers. I aim to translate this method into a practical vision architecture.

Name: T. Satya Pranav | **Roll No:** 2201213 | **Supervisor:** Dr. Kaustuv Nag

Literature review:

1. Attention Is All You Need (The Transformer):

- **Concept**: Introduced the standard Transformer architecture based on self-attention, removing recurrence entirely .
- **Gaps**:
  - **Parameter-Heavy**: Uses unique weights for every layer, leading to massive model sizes.
  - **Uniform Depth**: Every token must pass through the same fixed number of layers, regardless of complexity.

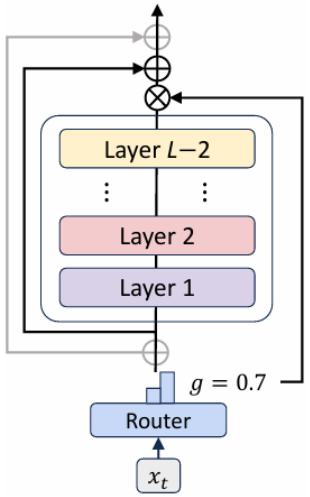


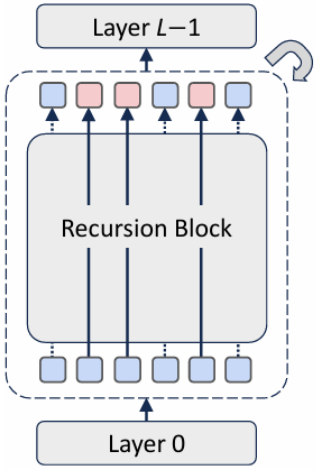
2. Relaxed Recursive Transformers (RRT)Concept:

- Introduced "layer tying" (recursion) to reuse a single block multiple times, reducing parameter count .
- **Gaps**:
  - **No Adaptive Computation**: While it saves parameters, it still forces every token to go through the same number of recursion steps. It lacks a mechanism to save compute for simple tokens.

3. Mixture-of-Recursions (MoR) :

- **Concept:** Unifies Parameter Sharing with Adaptive Computation. It defines a recursive update rule where a shared block $\boldsymbol{f}(\cdot;\boldsymbol{\Phi}')$ is applied repeatedly.
- **General Update Formula**: $\boldsymbol{h_t^{r+1}} = \boldsymbol{g_t^r} \cdot \boldsymbol{f}(\boldsymbol{h_t^r};\boldsymbol{\Phi}') + \boldsymbol{h_t^r}$ where $\boldsymbol{g_t^r}$ is the routing gate and Φ' are the shared parameters.
- **Bridging the Gap:** Solves "Uniform Depth" by allowing g_t^r to determine if a token continues processing or exits..
- **Application**: Adopted MoR as a proof-of-concept and apply this Token Removal strategy to Vision Transformers (ViT) to eliminate redundancy in image processing.



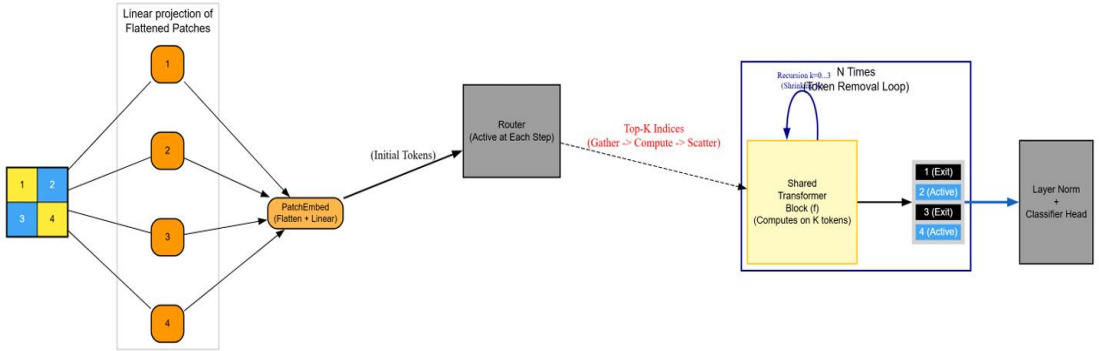


Work Done till date:

Details of proposed work/solutions:

- **Recursive Backbone**: Replaced the 8-layer stack with a single shared Transformer Encoder Block Φ') reused for 4 recursion steps.
- **Efficient Router**: Implemented an Expert-Choice router. At each step **r**, it computes an importance score $\boldsymbol{s_t^{(r)}}$ for token **t** using a sigmoid activation: $$\boldsymbol{s_t^{(r)} = \sigma\left(W_r \cdot h_t^{(r)} + b_r\right)}$$
- **Token Removal Mechanism**: The number of tokens kept, K_r, shrinks geometrically based on the keep rate $\boldsymbol{\beta = 0.7}$: $\boldsymbol{K_r = \lfloor N \times \beta^r \rfloor}$ Only the top-K_r tokens are gathered and processed ($\boldsymbol{h \in R^{K_r \times D}}$), effectively reducing FLOPs
- **Training Strategy:** Trained end-to-end using Cross-Entropy loss ($\boldsymbol{L_{task}}$) and an auxiliary Z-loss to stabilize the router's $(\boldsymbol{W_r h + b_r})$

$$\boldsymbol{L_{total} = L_{task} + \lambda \frac{1}{B} \sum_{i=1}^{B} \left(\log\left(\sum_{t=1}^{N} \exp\left(W_r \cdot h_{t,i}^{(r)} + b_r\right)\right)\right)^2}$$

Result achieved:

- **Parameter Reduction:** 85% reduction (25M →3.6M parameters).
- **Accuracy Improvement:** +0.87% Top-1 Accuracy (74.14% → 75.01%).
- **Ablation (Masked vs. Efficient):**
  - **Masked Routing:** 77.27% accuracy (No FLOP savings).
  - **Efficient Token Removal**: 75.01% accuracy (Significant FLOP savings).